

Project 2: Robot Wall Following by Reinforcement Learning

CSC-591: Advanced Robotics

Sushant Khattar
North Carolina State University
Raleigh, NC, 27695

Abstract—This paper presents a Q-learning approach for enabling a simulated omnidirectional robot (Triton) to perform wall following and navigate basic environmental structures. The robot uses 2D LiDAR data to represent its environment, and Q-learning is employed to train the robot to follow walls, turn corners, and avoid obstacles. The state space is defined by discretizing the robot’s angular error relative to a desired wall position, while actions correspond to discrete angular velocities. The reward function encourages maintaining a desired distance from the wall and penalizes collisions. Experimental results demonstrate the robot’s ability to learn effective wall-following behavior in simulation, including straight wall following and turning at L-shaped corners. The convergence rate of the Q-learning algorithm is analyzed, and qualitative results from video demonstrations showcase the learned behaviors.

I. INTRODUCTION

Wall following is a fundamental capability for autonomous robots, enabling navigation in unknown environments. This paper addresses the wall-following problem using reinforcement learning, specifically Q-learning, to train a simulated Triton robot to navigate a predefined environment in Gazebo. Unlike traditional control-rule based methods, Q-learning allows the robot to learn an optimal policy through interaction with the environment. This section provides an overview of the problem, the chosen approach, and a summary of the experimental results.

II. APPROACH

Our approach utilizes Q-learning, a temporal-difference reinforcement learning algorithm, to train the Triton robot for wall following. The key components of our implementation are detailed below:

A. State Representation

The state representation is a crucial component of our reinforcement learning approach. We chose to define states based on the angular error between the robot’s current heading and the desired heading toward the wall. This approach was inspired by the Stanley method for path tracking, which is commonly used in autonomous vehicles. The state space is defined based on the robot’s LiDAR readings. We use the angle error (θ_{err}) between the robot’s current heading and the direction towards a desired point along the wall [1]. θ_{err} is discretized into six states:

- State 1: $120^\circ < \theta_{err} \leq 180^\circ$
- State 2: $80^\circ < \theta_{err} \leq 120^\circ$

- State 3: $40^\circ < \theta_{err} \leq 80^\circ$
- State 4: $20^\circ < \theta_{err} \leq 40^\circ$
- State 5: $5^\circ < \theta_{err} \leq 20^\circ$
- State 6: $0^\circ \leq \theta_{err} \leq 5^\circ$

This discretization allows the robot to react appropriately based on its orientation relative to the wall.

B. Stanley Method for Wall Following

The Stanley method, originally developed for the Stanford autonomous vehicle that won the DARPA Grand Challenge [2], provides a simple but effective control law for path following. We adapted this approach for wall following as follows:

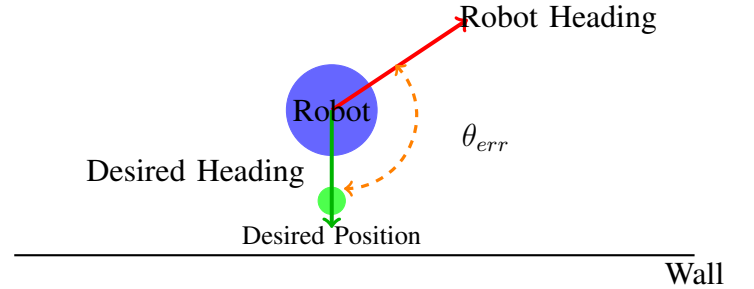


Fig. 1. Illustration of the Stanley method for wall following. The robot determines its position relative to the wall and calculates the angular error θ_{err} between its current heading and the desired heading toward a point at a fixed distance from the wall.

Using LiDAR data, the robot determines the closest point on the wall. A desired position is calculated at a fixed distance (0.4m) from this point. The angle error (θ_{err}) is computed between the robot’s current heading and the vector pointing from the robot to this desired position. The robot adjusts its steering to minimize this error.

The method can be expressed mathematically as:

$$\theta_{des} = \arctan\left(\frac{d_{stan}}{d_{min} - d_{des}}\right)$$

Where:

- θ_{des} is the desired heading angle
- d_{stan} is a parameter that controls how aggressively the robot approaches the desired path (set to 0.2m)
- d_{min} is the minimum distance to the wall from LiDAR readings

- d_{des} is the desired distance to maintain from the wall (set to 0.4m)

Then, the error angle is calculated as:

$$\theta_{err} = |\theta_{des} - \theta_{robot}|$$

This error angle is then discretized into six distinct states to create a manageable state space for Q-learning. Our state classification balances granularity with learning efficiency—fewer states learn faster but with less precision, while more states provide finer control but require more training time.

C. Q-Table Initialization and Structure

The Q-table is structured as a 6x10 matrix, representing 6 states and 10 possible actions. We initialize the Q-values with small random values between -0.1 and 0.1 rather than zeros, which helps to break symmetry during learning and encourages initial exploration.

```
Q-table structure:
[
  [q(s0,a0), ..., q(s0,a9)], # State 0
  [q(s1,a0), ..., q(s1,a9)], # State 1
  ...
  [q(s5,a0), ..., q(s5,a9)] # State 5
]
```

D. Action Space

The action space consists of ten discrete angular velocities (ω_z) that the robot can apply:

- Action 1: $\omega_z = 0^\circ/s$
- Action 2: $\omega_z = 15^\circ/s$
- Action 3: $\omega_z = 30^\circ/s$
- Action 4: $\omega_z = 45^\circ/s$
- Action 5: $\omega_z = 60^\circ/s$
- Action 6: $\omega_z = 75^\circ/s$
- Action 7: $\omega_z = 90^\circ/s$
- Action 8: $\omega_z = 110^\circ/s$
- Action 9: $\omega_z = 130^\circ/s$
- Action 10: $\omega_z = 150^\circ/s$

The robot chooses an action at each time step to adjust its heading.

E. Enhanced Reward Function Design

Our reward function is designed to encourage the robot to maintain the desired orientation with respect to the wall. The primary component of the reward is inversely proportional to the angular error:

$$R_{orientation} = \pi - |\theta_{err}|$$

This formulation provides a maximum reward of π (approximately 3.14) when the error is zero and linearly decreases as the error increases. The reward becomes zero when the error reaches π radians (180°), which represents the maximum possible error.

Additionally, we incorporate a significant negative reward (-50) when the robot gets too close to a wall (collision

penalty). This discourages the robot from colliding with walls and encourages it to maintain a safe distance.

The combination of these reward components creates a learning environment where the robot is incentivized to:

- Align itself properly with respect to the wall
- Maintain a consistent distance from the wall
- Avoid collisions

F. Q-Learning Implementation

We implemented the Q-learning algorithm as described in [3]. The Q-table is initialized with small random values. The Q-value update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where:

- $Q(s, a)$ is the Q-value for state s and action a
- α is the learning rate (set to 0.2)
- R is the reward received after taking action a in state s
- γ is the discount factor (set to 0.9)
- s' is the next state
- a' is the action that maximizes $Q(s', a')$

We implemented an ϵ -greedy policy for action selection, which balances exploration and exploitation. With probability ϵ , the robot chooses a random action (exploration), and with probability $1 - \epsilon$, it selects the action with the highest Q-value (exploitation).

The exploration rate ϵ starts at 0.9 to encourage extensive exploration in the early stages of learning and gradually decays by a factor of 0.998 after each episode, until it reaches a minimum value of 0.05. This decay schedule ensures that the robot continues to explore new actions throughout training but increasingly exploits its learned knowledge as training progresses.

G. Parameter Tuning

The following parameters were used for our Q-learning implementation:

- Learning rate (α): 0.2
- Discount factor (γ): 0.9
- Initial exploration rate (ϵ): 0.9
- Minimum exploration rate (ϵ_{min}): 0.05
- Exploration decay rate: 0.998
- Number of training episodes: 500

These parameters were chosen to balance exploration and exploitation during training.

H. Training Process Optimization

To enhance the training process, we implemented several optimizations:

- 1) **Dynamic Starting Positions:** The robot starts from different positions in the environment for each episode, which helps it generalize its learned policy across various scenarios.
- 2) **Episode Termination Conditions:** An episode ends after a maximum of 400 steps or when a collision occurs. This limit prevents excessively long episodes

and allows the robot to learn from a diverse set of experiences.

- 3) **Incremental Learning:** The Q-values are updated incrementally using the temporal difference (TD) method, which allows the robot to learn from individual steps rather than waiting until the end of an episode.
- 4) **Learning Parameters Tuning:** We carefully tuned the learning rate ($\alpha = 0.2$) and discount factor ($\gamma = 0.9$) to balance immediate and future rewards. The learning rate of 0.2 allows for relatively fast learning while maintaining stability, and the discount factor of 0.9 emphasizes long-term rewards without completely ignoring immediate feedback.

III. EXPERIMENTAL RESULTS

We evaluated our Q-learning implementation in the Gazebo simulation environment. The robot was trained for 500 episodes, and its performance was assessed based on the accumulated reward and its ability to follow walls and navigate corners.

A. Quantitative Analysis of Learning Performance

Figure 2 shows the accumulated reward per episode during training. The plot demonstrates that the Q-learning algorithm converges over time, as the accumulated reward generally increases with the number of episodes.

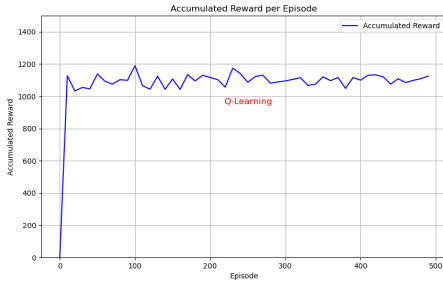


Fig. 2. Accumulated reward per episode during Q-learning.

The plot reveals several important aspects of the learning dynamics:

- **Initial Volatility:** During the first 100-150 episodes, the accumulated reward shows high volatility, indicating that the robot is primarily exploring the environment and encountering various states and transitions.
- **Increasing Trend:** After approximately 150 episodes, a general upward trend becomes apparent, suggesting that the robot is beginning to learn effective wall-following behaviors. The increased average reward indicates improved performance.
- **Continued Fluctuations:** Even as training progresses, the reward continues to fluctuate from episode to episode. These fluctuations can be attributed to the ϵ -greedy policy, which occasionally selects exploratory actions, and the varying difficulty of different starting positions.

- **Convergence Indication:** By episodes 400-500, the fluctuations in reward appear to stabilize around a higher average value, suggesting that the Q-learning algorithm is converging toward an optimal policy.

The total training time of 4.5 hours for 500 episodes aligns with expectations for reinforcement learning in this domain.

B. In-Depth Analysis of Learned Behaviors

Beyond the quantitative results, the following figures illustrate the robot's performance in different scenarios:

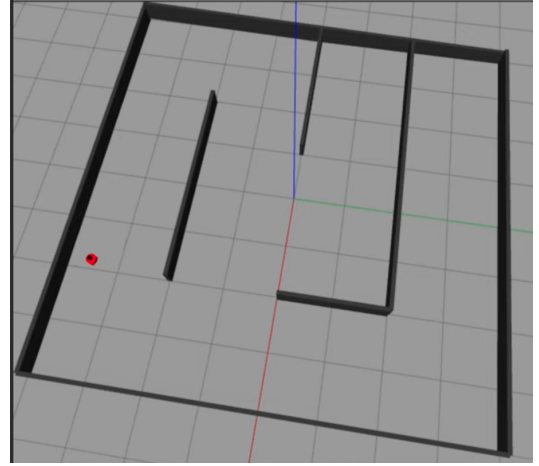


Fig. 3. Robot following a straight wall.

As shown in Figure 3, the robot maintains a consistent distance from the wall while moving forward, demonstrating its ability to follow a straight wall effectively.

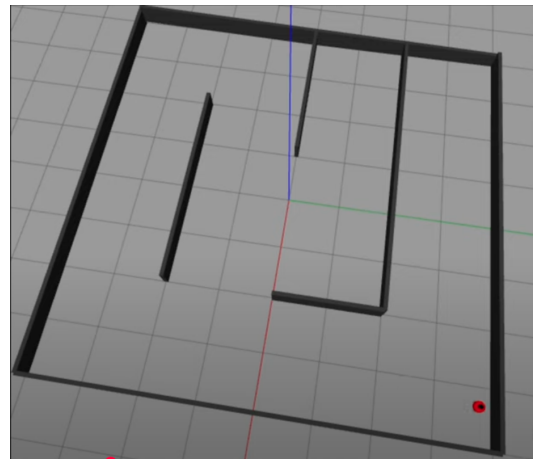


Fig. 4. Robot turning left 90 degrees at an L-shaped corner.

Figure 4 shows the robot successfully navigating a left 90-degree turn at an L-shaped corner. The robot adjusts its heading to align with the new wall orientation while maintaining its desired distance.

In Figure 5, the robot demonstrates its ability to execute a 180-degree turn at a U-shaped corner, effectively reversing its direction while staying close to the walls.

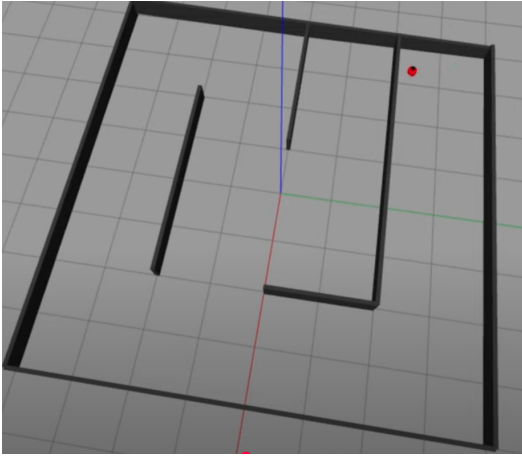


Fig. 5. Robot turning 180 degrees at a U-shaped corner.

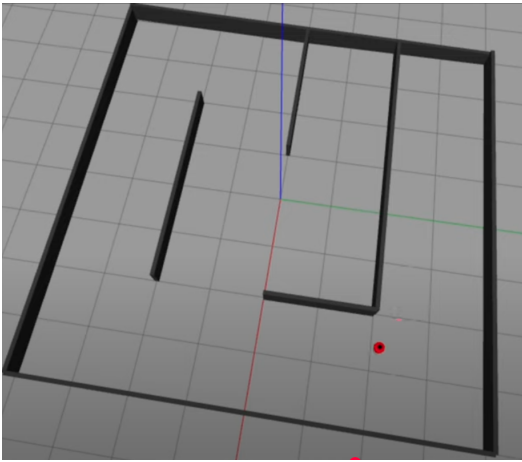


Fig. 6. Robot turning right 90 degrees at an L-shaped corner.

Figure 6 illustrates the robot performing a right 90-degree turn at an L-shaped corner, showcasing its ability to adapt to different turning directions.

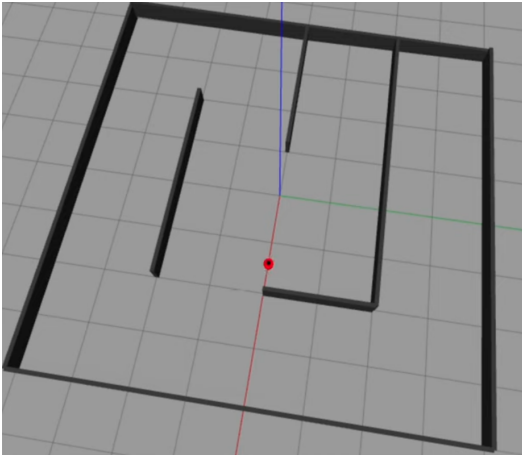


Fig. 7. Robot turning 180 degrees at an I-shaped corner.

Finally, Figure 7 shows the robot navigating a 180-degree turn at an I-shaped corner, demonstrating its capability to

handle complex turns in narrow spaces.

We observed several interesting qualitative behaviors that emerged during the robot's training:

Adaptive Corner Navigation: The robot demonstrated the ability to navigate corners of different types, as shown in Figures 4, 5, 6, and 7. For 90-degree turns, the robot adjusts its angular velocity gradually as it approaches the corner, then increases the turning rate to align with the new wall direction. For 180-degree turns, the robot executes a tighter turning radius and maintains a more consistent angular velocity throughout the turn.

Distance Maintenance: As shown in Figure 3, the robot successfully learns to maintain a consistent distance from the wall while moving. This behavior emerges from the reward function's emphasis on minimizing angular error relative to the desired wall position.

Recovery Behaviors: An interesting emergent behavior is the robot's ability to recover from suboptimal positions. When the robot finds itself too far from or too close to the wall, it quickly adjusts its trajectory to return to the desired distance.

These results highlight the effectiveness of the Q-learning algorithm in enabling the robot to handle various wall-following and corner-turning scenarios.

IV. CONCLUSION

The Q-learning approach enables robot wall following in a simulated environment. The robot successfully learns to follow walls and navigate corners. The discretization of the state space based on angular error proved to be an effective representation for the wall-following task. The six-state discretization provided sufficient granularity while maintaining reasonable learning efficiency. The reward function based on angular error effectively guided the learning process toward desired behaviors. The balance between positive rewards for correct orientation and negative penalties for collisions was crucial for successful learning.

A. Limitations and Challenges

Despite the overall success, several limitations and challenges were observed:

- **Discretization Limitations:** The discrete state and action spaces limit the precision of the robot's control. A finer discretization could potentially improve performance but would require more training time.
- **Generalization to New Environments:** While the robot performed well in the training environment, its performance in significantly different environments remains to be evaluated.
- **Optimization of Parameters:** Manual tuning of hyperparameters was necessary to achieve good performance.

B. Future Work

Several promising directions for future work include:

- **Continuous State and Action Spaces:** Implementing more advanced reinforcement learning algorithms that

support continuous state and action spaces, such as Deep Q-Networks (DQN) or Policy Gradient methods.

- **Real-World Implementation:** Testing the learned policies on physical robots would reveal practical challenges and opportunities for bridging the reality gap between simulation and real-world deployment.

REFERENCES

- [1] C. Duncan, "Wall-Following Robot with Reinforcement Learning," GitHub repository, 2023. [Online]. Available: <https://github.com/cduncan20/Wall-Following-Robot-with-RL>
- [2] S. Thrun, M. Montemerlo, H. Dahlkamp, D. Stavens, A. Aron, J. Diebel, P. Fong, J. Gale, M. Halpenny, G. Hoffmann, K. Lau, C. Oakley, M. Palatucci, V. Pratt, P. Stang, S. Strohband, C. Dupont, L. Jendrossek, C. Koelen, C. Markey, C. Rummel, J. van Niekerk, E. Jensen, P. Alessandrini, G. Bradski, B. Davies, S. Ettinger, A. Kaehler, A. Nefian, and P. Mahoney, "Stanley: The Robot that Won the DARPA Grand Challenge," *Journal of Field Robotics*, vol. 23, no. 9, pp. 661-692, 2006.
- [3] Project 2: Robot Wall Following by Reinforcement Learning, CSC-591: Advanced Robotics, 2025.